Memoria del taller: Procesado y análisis de datos con Python

Laboratorio de Proyectos Tecnológicos

Ismael García López José Luis Vega García

27 de abril de 2026

Índice

1. Introducción **2**

2. Fase 1: Lectura, procesado y visualización en tiempo real de datos de una red **2**

2.1. Punto 1: Lectura e Identificación en el Log 2

2.2. Puntos 2 y 3: Identificación y almacenamiento temporal de volúmenes SDAP 3

2.3. Punto 4: Agregación cada 1 segundo y volcado a DataFrames 4

2.4. Punto 5: Representación en tiempo real de la evolución de volúmenes con Dash 5

2.5. Punto 6: Almacenamiento en formato Parquet cada 30 segundos 6

3. Fase 2: Identificación de Identidades de Red **6**

3.1. Punto 1: Extracción de parámetros de red y celdas 6

4. Conclusiones **7**

1. Introducción

En este documento se detalla el desarrollo del taller 4 correspondiente a la asignatura Laboratorio de Proyectos Tecnológicos, implementada en Python. El objetivo del proyecto se divide en diferentes fases centradas en el procesado y análisis de datos procedentes de los logs de una red simulada.

2. Fase 1: Lectura, procesado y visualización en tiempo real de datos de una red

El objetivo de la primera fase consiste en leer en tiempo real un log generado por una Central Unit (CU), identificar los mensajes con información de volumen de datos transferido a nivel SDAP, procesarlos mediante DataFrames y representarlos gráficamente.

2.1. Punto 1: Lectura e Identificación en el Log

El primer reto consiste en implementar un mecanismo que permita leer un archivo de registro (log) mientras este está siendo escrito por otro proceso (el simulador de la CU). A diferencia de una lectura convencional, necesitamos una lectura de tipo *streaming*.

Para lograr la monitorización en vivo, el sistema implementa un bucle infinito. Sin embargo, debido a que los logs pueden contener líneas incompletas, basura o metadatos del sistema, se ha implementado un filtro de restricción estricta basado en expresiones regulares para asegurar que solo se procesen eventos válidos que empiecen con formato de línea temporal.

```python
def motor_lectura():
    global historico_grafica, datos_totales_sesion, info_usuarios

    inicio_segundo = None
    proximo_guardado_parquet = None
    datos_segundo_actual = []

    # Expresion regular para validar que la linea empieza con fecha ISO
    # Ejemplo: 2026-01-19T08:55:48.084727
    patron_fecha = re.compile(r"^(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d+)")

    with open(archivo_log, 'r') as archivo:
        while True:
            linea = archivo.readline()
            if not linea:
                time.sleep(0.01)
                continue

            # 1. RESTRICCION ESTRICTA: Validacion mediante Regex
            match_fecha = patron_fecha.match(linea)
            if not match_fecha:
                continue # Ignoramos lineas sin formato de fecha valido

            # Extraccion segura del tiempo usando el grupo capturado
            tiempo_str = match_fecha.group(1)
```

```
            tiempo_actual_log = datetime.fromisoformat(tiempo_str)
```

Listing 1: Lectura con validacion estricta de fecha ISO

Explicación del código:

El uso del objeto `patron_fecha` compilado mediante `re.compile()`, a diferencia de un simple `split()`, garantiza que el motor de lectura solo procese líneas que cumplan el formato de logging de la CU.

El uso de la expresión regular `r"^(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d+)"` se desglosa de la siguiente manera:

- **Ancla de inicio** `^`: Obliga a que el patrón coincida desde el primer carácter de la línea. Si hay espacios o texto basura previo, la línea se descarta.
- **Grupo de captura** `(...)`: Encierra toda la estructura de la fecha para que podamos extraerla directamente con `match_fecha.group(1)` sin necesidad de manipular la cadena manualmente.
- **Cuantificadores de dígitos** `\d{n}`: Valida la estructura exacta del calendario (4 dígitos para el año, 2 para mes, etc.).
- **Literales de formato** (`T`, `-` y `:`): Aseguran que el separador ISO y los separadores de tiempo estén presentes en sus posiciones correctas.
- **Escape del punto** `\.`: El punto es un carácter especial en Regex; al escaparlo, indicamos que buscamos el punto decimal que precede a los microsegundos.

2.2. Puntos 2 y 3: Identificación y almacenamiento temporal de volúmenes SDAP

Una vez que la línea ha sido leída, el sistema debe discernir si contiene información relevante sobre el tráfico de datos en el enlace de descarga (DL) a nivel SDAP. Al mismo tiempo, para procesar esta información en tiempo real sin bloquear el sistema, los datos extraídos se almacenan instantáneamente en un búfer en memoria.

```
            # --- FASE 1: FILTRADO SDAP (Descarga de datos) ---
            # Identificamos lineas con el protocolo SDAP y la accion TX PDU
            if "SDAP" in linea and "TX PDU" in linea:
                # Extraemos el identificador del UE y la longitud de la PDU (pdu_len)
                match_datos = re.search(r"ue=(\d+).*pdu_len=(\d+)", linea)

                if match_datos:
                    # Guardamos en tiempo real los volumenes indicados para cada UE
                    datos_segundo_actual.append({
                        "ue": int(match_datos.group(1)),
                        "bytes": int(match_datos.group(2))
                    })
```

Listing 2: Filtrado SDAP y almacenamiento en búfer temporal

Explicación del código: En este bloque se combinan dos acciones. Primero (Punto 2), aplicamos un filtro lógico para procesar solo las líneas que contienen "SDAP" y "TX PDU", utilizando una expresión regular (`re.search`) para capturar el identificador del usuario (`ue`) y el tamaño del paquete en bytes (`pdu_len`). Segundo (Punto 3), encapsulamos estos valores inmediatamente en un diccionario y los añadimos a la lista `datos_segundo_actual`. Esta lista de Python actúa como un búfer, permitiendo absorber miles de registros por segundo listos para su posterior agregación matemática.

2.3. Punto 4: Agregación cada 1 segundo y volcado a DataFrames

El núcleo del procesado analítico reside en consolidar el tráfico de red de forma periódica. Para ello, cada segundo se convierte el búfer temporal en un DataFrame de Polars y se realiza una suma agregada para obtener el consumo total, guardando los resultados en una nueva estructura consolidada.

```
            # FASE 1: AGREGACION CADA 1 SEGUNDO
            if (tiempo_actual_log - inicio_segundo).total_seconds() >= 1.0:
                if datos_segundo_actual:
                    # 1. Convertimos la lista a DataFrame
                    df = pl.DataFrame(datos_segundo_actual)

                    # 2. Agrupamos por 'ue' y sumamos los bytes
                    resumen = df.group_by("ue").agg(pl.col("bytes").sum().alias("consumo"))

                    # 3. Guardado seguro en la estructura historica
                    with lock:
                        for fila in resumen.to_dicts():
                            ue_id = fila['ue']

                            # ... info_usuarios ...

                            dato = {
                                "Tiempo": inicio_segundo,
                                "UE": etiqueta_pro,
                                "SDAP (Bytes)": fila['consumo']
                            }
                            # Guardamos en otro DataFrame/Lista consolidado
                            historico_grafica.append(dato)
                            datos_totales_sesion.append(dato)
```

Listing 3: Agregación de datos por segundo mediante Polars

Explicación del código: Evaluamos si la diferencia entre el tiempo actual del log y el `inicio_segundo` es mayor o igual a 1.0 segundos. Si es así, volcamos el búfer `datos_segundo_actual` en un `pl.DataFrame`. A continuación, utilizamos `group_by(''ue'')` seguido de `agg` para sumar todos los bytes de ese segundo bajo la nueva columna `consumo`.

Finalmente, iteramos sobre este resumen. Utilizamos un contexto `with lock` para evitar colisiones de lectura/escritura con el hilo que dibuja la gráfica, y estructuramos cada fila en un diccionario (`dato`) que se guarda en `historico_grafica` y `datos_totales_sesion`, limpiando después el búfer para el siguiente segundo.

2.4. Punto 5: Representación en tiempo real de la evolución de volúmenes con Dash

Para monitorizar la evolución del tráfico de cada usuario de forma dinámica, se ha implementado un servidor web local utilizando `Dash` y `Plotly`. El objetivo de este bloque es que la interfaz lea los datos agregados y se actualice automáticamente sin necesidad de que el usuario recargue la página del navegador.

```python
# --- INTERFAZ DASH ---
app = Dash(__name__)

# Definicion de la estructura web (Layout)
app.layout = html.Div([
    html.H1("Monitor de Tr fico 5G (Fases 1 y 2 Completadas)", style={'textAlign': 'center'}),
    dcc.Graph(id='grafica-trafico'),
    dcc.Interval(id='intervalo-actualizacion', interval=1000, n_intervals=0)
])

# Funcion de re-dibujado que se ejecuta cada segundo
@app.callback(
    Output('grafica-trafico', 'figure'),
    Input('intervalo-actualizacion', 'n_intervals')
)
def actualizar_grafica(n):
    # Bloqueamos el acceso para asegurar una lectura coherente
    with lock:
        if not historico_grafica:
            return px.line(title="Esperando flujo de datos...")

        # Convertimos el historico a DataFrame y ordenamos cronologicamente
        df_plot = pl.DataFrame(historico_grafica).sort("Tiempo")

    # Generamos la grafica de lineas separando cada UE por color
    return px.line(df_plot, x="Tiempo", y="SDAP (Bytes)", color="UE", markers=True)
```

Listing 4: Implementación del dashboard interactivo con Dash

Explicación del código: La aplicación web se define en `app.layout`, donde se incluyen un título, el contenedor vacío de la gráfica (`dcc.Graph`) y, lo más importante, un temporizador invisible (`dcc.Interval`) configurado para dispararse cada segundo. Este temporizador es el que llama a la función `actualizar_grafica` mediante `@app.callback`.

Dentro de la función de actualización, volvemos a utilizar el `lock` para garantizar que no haya colisiones de lectura mientras el motor secundario escribe nuevos datos. Una vez copiada la lista `historico_grafica` en un DataFrame de Polars estructurado y ordenado, utilizamos `px.line` de `Plotly Express`. Al asignarle el parámetro `color=''UE''`, la librería se encarga automáticamente de dibujar una línea distinta para cada usuario registrado, mostrando sus picos de consumo de bytes frente al tiempo real.

2.5. Punto 6: Almacenamiento en formato Parquet cada 30 segundos

Para permitir análisis históricos posteriores, el sistema vuelca los datos acumulados durante la sesión en un archivo de formato Apache Parquet de forma periódica.

```
            # FASE 1: GUARDADO EN PARQUET
            # Comprobamos si ha llegado el momento del guardado periodico
            if tiempo_actual_log.timestamp() >= proximo_guardado_parquet:
                if datos_totales_sesion:
                    # Creamos un DataFrame con todo lo acumulado en la sesion
                    df_parquet = pl.DataFrame(datos_totales_sesion)
                    # Escribimos el archivo en disco (formato eficiente)
                    df_parquet.write_parquet(archivo_parquet)

                # Programamos el proximo guardado para dentro de 30 segundos
                proximo_guardado_parquet = tiempo_actual_log.timestamp() + 30
```

Listing 5: Volcado periodico de datos a archivo Parquet

Explicación del código: Utilizamos una variable de control llamada `proximo_guardado_parquet` que se inicializa al empezar y se incrementa en pasos de 30 segundos. Cuando el tiempo interno del log alcanza este umbral, convertimos la lista global `datos_totales_sesion` en un DataFrame y lo exportamos.

3. Fase 2: Identificación de Identidades de Red

Esta fase se centra en la extansión de los datos capturados. Mientras que la Fase 1 se encarga del volumen de tráfico, la Fase 2 informa de más datos sobre el usuario, como el PCI (*Physical Cell Identity*, identificador físico de celda), el PLMN (*Public Land Mobile Network*, código de identificación mundial del operador de telefonía) y el RNTI (*Radio Network Temporary Identifier*, identificador de red radio temporal).

3.1. Punto 1: Extracción de parámetros de red y celdas

Para identificar la identidad de cada usuario, el sistema monitoriza el log en busca del evento de creación de un nuevo UE en el plano de control (CU-CP). La línea de información típica en la que viene recogida toda esta información tiene el siguiente formato:

```
2026-01-19T08:55:48.084727 [CU-UEMNG] [I] ue=1 du_index=2 plmn=21405 gnb_du_id=2 pci=802 rnti=0x4602 pcell_index=1: Created new CU-CP UE
```

Listing 6: Linea de log con informacion de red

A partir de este formato, hemos implementado una lógica de extracción mediante expresiones regulares para capturar los campos necesarios y almacenarlos en un diccionario global.

```
# --- FASE 2: CAZADOR DE IDENTIDADES ---
# Buscamos la linea de creacion del UE para nutrir la agenda
if "Created new CU-CP UE" in linea:
```

```python
    # Utilizamos Regex para capturar UE ID, PLMN, PCI y RNTI
    match_info = re.search(r"ue=(\d+).*?plmn=(\d+).*?pci=(\d+).*?rnti=(\w+)", linea)

    if match_info:
        ue_id = int(match_info.group(1))

        # Guardamos los datos en el diccionario 'info_usuarios'
        info_usuarios[ue_id] = {
            "plmn": match_info.group(2),
            "pci": match_info.group(3),
            "rnti": match_info.group(4)
        }
```

Listing 7: Cazador de identidades y agenda de usuarios

Explicación del código: El script utiliza la condición `''Created new CU-CP UE'' in linea` para activar el motor de extracción solo en los momentos de alta de usuarios, optimizando el rendimiento.

La expresión regular empleada utiliza el cuantificador `.*?` (non-greedy) para saltar de forma eficiente entre los campos del log sin importar cuánta información intermedia haya (como `du_index` o `gnb_du_id`). Capturamos cuatro grupos: el ID del usuario, el código de red (**PLMN**), el identificador de celda física (**PCI**) y el identificador temporal de red de radio (**RNTI**). Estos datos se indexan por el `ue_id` en un diccionario global, lo que permite que el Dashboard consulte esta información instantáneamente cada vez que se detecta tráfico de datos para ese usuario.

4. Conclusiones

El desarrollo de esta práctica ha permitido consolidar los conocimientos sobre el procesado y análisis de datos en tiempo real aplicados a redes de telecomunicaciones, demostrando la utilidad de herramientas de Python como `Polars` y `Dash`.

Tal y como se observa en la Figura 1, el dashboard interactivo logra integrar con éxito los objetivos de todas las fases del proyecto, ofreciendo una representación clara de la evolución temporal del volumen de datos a nivel SDAP.

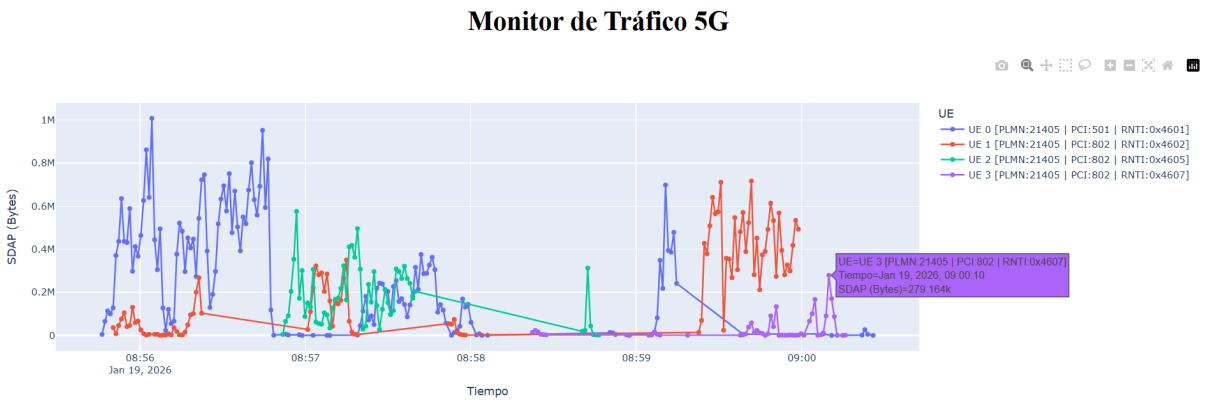

Figura 1: Monitorización en Dash: Evolución temporal del tráfico SDAP por cada UE.

A partir del análisis visual de la gráfica resultante, se pueden extraer las siguientes conclusiones sobre el comportamiento del tráfico en la red 5G simulada:

- **Identificación:** La leyenda de la gráfica evidencia el éxito de la extracción mediante expresiones regulares. Cada curva no solo identifica al usuario (UE 0, UE 1, UE 2, UE 3), sino que expone en tiempo real su contexto de red extraído del plano de control (PLMN 21405, la celda física PCI y el RNTI asignado). Como dato curioso, el código 214 indica país España, y 05 indica que se trata de la compañía Movistar.

- **Dinámica Temporal por Intervalos:** Se observa claramente que el tráfico en la red no es constante ni simultáneo para todos los equipos. Los usuarios inician y finalizan sus sesiones en diferentes ventanas temporales. Por ejemplo, el UE 0 (línea azul) presenta una alta actividad al inicio de la captura (08:56), seguido por una ráfaga del UE 1 (naranja) y con otra del UE 2 (verde) sobre las 08:57. Más adelante, hacia las 08:59 y 09:00, son el UE 0 (azul), UE 1 (rojo) y el UE 3 (morado) los que acaparan los recursos de la red.

- **Variaciones en el Consumo SDAP:** Las curvas reflejan un comportamiento de tráfico a ráfagas, característico de las redes móviles. Se aprecian fuertes variaciones que oscilan desde la inactividad total (0 Bytes) hasta picos máximos que alcanzan el megabyte.

En definitiva, la solución diseñada cumple con todos los requisitos exigidos: es capaz de leer un flujo continuo de logs de forma no bloqueante, estructurar y cruzar identidades con volúmenes de datos.